

目录

- 1 作业内容**
- 2 Redis 简介**
- 3 实验环境**
- 4 实验步骤**
 - 4.1 Redis 源码安装及编译
 - 4.2 测试框架安装及说明
 - 4.3 测试过程
- 5 实验结果及分析**
 - 5.1 测试用例 #1: HSET/HLEN - Small hash creation
 - 5.2 测试用例 #2: Is the small hash encoded with a listpack?
 - 5.3 测试用例 #3: packed node check compression with insert and pop
 - 5.4 测试用例 #4: SADD, SCARD, SISMEMBER, SMISMEMBER, SMEMBERS basics - listpack
 - 5.5 测试用例 #5: HRANDFIELD - listpack
 - 5.6 测试用例 #6: HSET/HLEN - Big hash creation
 - 5.7 测试用例 #7: SINTER with two sets - regular
 - 5.8 测试用例 #8: LPUSH, RPUSH, LLENGTH, LINDEX, LPOP - quicklist
 - 5.9 测试用例 #9: BLPOP: single existing list - quicklist
 - 5.10 测试用例 #10: SDIFF fuzzing
- 6 实验总结**
- 7 附录A: 完整测试运行结果**

1 作业内容

安装 redis，并运行 redis 的自动测试脚本：runtest，从运行结果中任意选出 10 条，找到对应的这 10 个测试用例在自动测试脚本中对应的代码。

2 Redis 简介

Redis（Remote Dictionary Server，远程字典服务）是一个开源（BSD 许可）的内存数据结构存储系统，常被用作数据库、缓存和消息代理。与传统的关系型数据库或磁盘存储系统不同，Redis 将数据存储于内存中，这使其能够以极快的速度（通常在亚毫秒级）执行读写操作。

3 实验环境

配置项	参数
操作系统	macOS Sequoia 15.6
CPU	Apple M3
Redis	8.2.2
TCL	8.6.14

4 实验步骤

4.1 Redis 源码安装及编译

由于 redis 的自动测试脚本 runtest 部分发行版未提供，所以这里选择安装源码进行编译的方式：

```
1 git clone https://github.com/redis/redis.git
2 cd redis
```

我选择切换到最新稳定版本（截止实验时最新版本为 8.2.2）而不是在 unstable 下进行：

```
1 # 切换到稳定版本
2 git checkout 8.2.2
```

源码仓库的 Makefile 提供了自动化编译选项：

```
1 # 清理旧的编译文件 (可选)
2 make distclean
3 # 重新编译
4 make
5 # 验证 Redis 已编译
6 ls -la src/redis-server # 已存在
```

在我的个人电脑上的执行结果如下：

```
LINK redis-server
INSTALL redis-sentinel
CC redis-cli.o
CC redisassert.o
CC cli_common.o
CC cli_commands.o
LINK redis-cli
CC redis-benchmark.o
LINK redis-benchmark
INSTALL redis-check-rdb
INSTALL redis-check-aof

Hint: It's a good idea to run 'make test' ;)

● > ls -la src/redis-server
-rwxr-xr-x 1 yokumi staff 2794592 10 21 18:27 src/redis-server
```

4.2 测试框架安装及说明

Redis 使用 TCL (Tool Command Language) 编写测试用例，包括：

- 测试脚本: `runtest` (Shell wrapper, 也可以通过 `make test`)
- 测试引擎: `tests/test_helper.tcl`
- 测试套件: `tests/unit/`, `tests/integration/` 等目录
- TCL 版本要求: 8.5, 8.6 或 8.7

macOS 上安装 TCL 的操作如下：

```
1 # 使用 Homebrew 安装 tcl-tk 包, 其中包含了 tclsh 解释器
2 brew install tcl-tk
3
4 # 进入 tclsh 环境验证是否可用
5 tclsh
```

安装成功并进入 Tcl shell 环境后，运行 `info patchlevel` 查看 TCL 版本如下：

```
> tclsh
% info patchlevel
8.6.14
% exit
```

4.3 测试过程

```
1 # 执行测试
2 ./runtest
```

为便于观察输出结果，这里我直接选择以下三个测试套件运行：

```
1 ./runtest --single unit/type/list \
2           --single unit/type/hash \
3           --single unit/type/set
```

运行，观察到以下结果：（完整输出结果见[附录A](#)）

```
The End

Execution time of different units:
 3 seconds - unit/type/hash
 7 seconds - unit/type/set
12 seconds - unit/type/list
 1 seconds - list-large-memory
 1 seconds - set-large-memory

\o/ All tests passed without errors!

Cleanup: may take some time... OK
```

其中，单元测试套件包含的测试用例输出格式为：

1	[ok]: 测试名称 (执行时间)
---	-------------------

示例：

1	[ok]: HSET/HLEN - Small hash creation (0 ms)
2	[ok]: SADD, SCARD, SISMEMBER, SMISMEMBER, SMEMBERS basics - listpack (1 ms)

5 实验结果及分析

我以 10 个典型的测试用例进行分析。

5.1 测试用例 #1: HSET/HLEN - Small hash creation

测试文件: `tests/unit/type/hash.tcl:5-18`

测试目的: 验证小型哈希表的创建和长度统计功能

测试代码:

1	test {HSET/HLEN - Small hash creation} {
2	array set smallhash {}
3	for {set i 0} {\$i < 8} {incr i} {
4	set key __avoid_collisions__[randstring 0 8 alpha]
5	set val __avoid_collisions__[randstring 0 8 alpha]
6	if {[info exists smallhash(\$key)]} {
7	incr i -1
8	continue
9	}
10	r hset smallhash \$key \$val
11	set smallhash(\$key) \$val
12	}
13	list [r hlen smallhash]
14	} {8}

功能说明:

- 创建包含 8 个随机键值对的哈希表
- 使用 `HSET` 命令插入数据
- 使用 `HLEN` 命令验证长度为 8

- 预期结果: 返回 `{8}`

执行结果:

```
1 [ok]: HSET/HLEN - Small hash creation (1 ms)
```

5.2 测试用例 #2: Is the small hash encoded with a listpack?

测试文件: `tests/unit/type/hash.tcl:20-22`

测试目的: 验证小型哈希表使用 listpack 编码优化

测试代码:

```
1 test {Is the small hash encoded with a listpack?} {  
2     assert_encoding listpack smallhash  
3 }
```

功能说明:

- 检查哈希表内部编码类型
- listpack 是 Redis 针对小数据量优化的紧凑编码
- 当元素数量和大小低于阈值时自动使用

执行结果:

```
1 [ok]: Is the small hash encoded with a listpack? (0 ms)
```

5.3 测试用例 #3: packed node check compression with insert and pop

测试文件: `tests/unit/type/list.tcl:26-44`

测试目的: 验证列表压缩节点的插入和弹出操作

测试代码:

```
1 test "packed node check compression with insert and pop" {  
2     r flushdb  
3     r lpush list1 [string repeat a $size]  
4     r lpush list1 [string repeat b $size]  
5     r lpush list1 [string repeat c $size]  
6     r lpush list1 [string repeat d $size]  
7     r linsert list1 after [string repeat d $size] [string repeat e $size]  
8     r linsert list1 after [string repeat d $size] [string repeat f $size]  
9     r linsert list1 after [string repeat d $size] [string repeat g $size]  
10    r linsert list1 after [string repeat d $size] [string repeat j $size]  
11    assert_equal [r lpop list1] [string repeat d $size]  
12    assert_equal [r lpop list1] [string repeat j $size]  
13    assert_equal [r lpop list1] [string repeat g $size]  
14    assert_equal [r lpop list1] [string repeat f $size]  
15    assert_equal [r lpop list1] [string repeat e $size]  
16    assert_equal [r lpop list1] [string repeat c $size]  
17    assert_equal [r lpop list1] [string repeat b $size]  
18    assert_equal [r lpop list1] [string repeat a $size]
```

功能说明:

- 测试压缩列表的 `LPUSH` 和 `LINSERT` 操作
- 验证 `LPOP` 弹出顺序的正确性
- 针对 packed node (500字节) 和 plain node (8193字节) 分别测试

执行结果:

```
1 [ok]: packed node check compression with insert and pop (1 ms)
```

5.4 测试用例 #4: SADD, SCARD, SISMEMBER, SMISMEMBER, SMEMBERS basics – listpack

测试文件: `tests/unit/type/set.tcl:21-36`

测试目的: 验证集合基本操作在 listpack 编码下的功能

测试代码:

```
1 test "SADD, SCARD, SISMEMBER, SMISMEMBER, SMEMBERS basics - $type" {
2     create_set myset $initelems($type)
3     assert_encoding $type myset
4     assert_equal 1 [r sadd myset bar]
5     assert_equal 0 [r sadd myset bar]
6     assert_equal [expr [llength $initelems($type)] + 1] [r scard myset]
7     assert_equal 1 [r sismember myset foo]
8     assert_equal 1 [r sismember myset bar]
9     assert_equal 0 [r sismember myset bla]
10    assert_equal {1} [r smismember myset foo]
11    assert_equal {1 1} [r smismember myset foo bar]
12    assert_equal {1 0} [r smismember myset foo bla]
13    assert_equal {0 1} [r smismember myset bla foo]
14    assert_equal {0} [r smismember myset bla]
15    assert_equal "bar $initelems($type)" [lsort [r smembers myset]]
16 }
```

功能说明:

- `SADD`: 添加元素 (重复添加返回0)
- `SCARD`: 获取集合大小
- `SISMEMBER`: 检查单个成员是否存在
- `SMISMEMBER`: 批量检查成员是否存在
- `SMEMBERS`: 获取所有成员

执行结果:

```
1 [ok]: SADD, SCARD, SISMEMBER, SMISMEMBER, SMEMBERS basics - listpack (1 ms)
```

5.5 测试用例 #5: HRANDFIELD – listpack

测试文件: `tests/unit/type/hash.tcl:49-58`

测试目的: 验证哈希表随机字段获取功能

测试代码:

```
1 test "HRANDFIELD - $type" {
2     unset -nocomplain myhash
3     array set myhash {}
4     for {set i 0} {$i < 100} {incr i} {
5         set key [r hrandfield myhash]
6         set myhash($key) 1
7     }
8     assert_equal [lsort [get_keys $contents]] [lsort [array names myhash]]
9 }
```

功能说明:

- 随机获取 100 次字段，统计分布
- 验证所有字段都有机会被获取到
- 确保随机性和完整性

执行结果:

```
1 [ok]: HRANDFIELD - listpack (7 ms)
```

5.6 测试用例 #6: HSET/HLEN – Big hash creation

测试文件: `tests/unit/type/hash.tcl:149-159`

测试目的: 验证大型哈希表创建（触发编码转换）

测试代码:

```
1 test {HSET/HLEN - Big hash creation} {
2     array set bighash {}
3     for {set i 0} {$i < 1024} {incr i} {
4         set key __avoid_collisions__[randstring 0 8 alpha]
5         set val __avoid_collisions__[randstring 0 8 alpha]
6         if {[info exists bighash($key)]} {
7             incr i -1
8             continue
9         }
10        r hset bighash $key $val
11        set bighash($key) $val
12    }
13    list [r hlen bighash]
14 } {1024}
```

功能说明:

- 创建 1024 个元素的大哈希表
- 当超过 `hash-max-listpack-entries` (512) 时自动转为 hashtable 编码

- 验证大数据量下的正确性

执行结果:

```
1 [ok]: HSET/HLEN - Big hash creation (35 ms)
```

5.7 测试用例 #7: SINTER with two sets – regular

测试文件: `tests/unit/type/set.tcl:312-314`

测试目的: 验证两个集合的交集操作

测试代码:

```
1 test "SINTER with two sets - $type" {
2     assert_equal [list 195 196 197 198 199 $large] [lsort [r sinter set1{t}
3     set2{t}]]
4 }
```

功能说明:

- `SINTER key1 key2`: 返回多个集合的交集
- 测试 regular (hashtable) 编码类型
- 验证集合运算的正确性

执行结果:

```
1 [ok]: SINTER with two sets - regular (0 ms)
```

5.8 测试用例 #8: LPUSH, RPUSH, LLENGTH, LINDEX, LPOP – quicklist

测试文件: `tests/unit/type/list.tcl:554+`

测试目的: 验证列表基本操作在 quicklist 编码下的功能

测试代码:

```
1 test "LPUSH, RPUSH, LLENGTH, LINDEX, LPOP - $type" {
2     # first lpush then rpush
3     r del mylist1
4     assert_equal 1 [r lpush mylist1 $large]
5     assert_encoding $type mylist1
6     assert_equal 2 [r rpush mylist1 b]
7     assert_equal 3 [r rpush mylist1 c]
8     assert_equal 3 [r llen mylist1]
9     assert_equal $large [r lindex mylist1 0]
10    assert_equal b [r lindex mylist1 1]
11    assert_equal c [r lindex mylist1 2]
12    assert_equal {} [r lindex mylist1 3]
13    assert_equal c [r rpop mylist1]
14    assert_equal $large [r lpop mylist1]
15
16    # first rpush then lpush
17    r del mylist2
```



```

18     assert_equal 1 [r rpush mylist2 $large]
19     assert_equal 2 [r lpush mylist2 b]
20     assert_equal 3 [r lpush mylist2 c]
21     assert_encoding $type mylist2
22     assert_equal 3 [r llen mylist2]
23     assert_equal c [r lindex mylist2 0]
24     assert_equal b [r lindex mylist2 1]
25     assert_equal $large [r lindex mylist2 2]
26     assert_equal {} [r lindex mylist2 3]
27     assert_equal $large [r rpop mylist2]
28     assert_equal c [r lpop mylist2]
29 }

```

功能说明:

- `LPUSH`: 从左侧插入元素
- `RPUSH`: 从右侧插入元素
- `LLENGTH`: 获取列表长度
- `LINDEX`: 按索引获取元素
- `LPOP`: 从左侧弹出元素

执行结果:

```

1 [ok]: LPUSH, RPUSH, LLENGTH, LINDEX, LPOP - quicklist (1 ms)

```

5.9 测试用例 #9: BLPOP: single existing list – quicklist

测试文件: `tests/unit/type/list.tcl:661+`

测试目的: 验证阻塞式列表弹出操作

测试代码:

```

1 test "$pop: single existing list - $type" {
2     set rd [redis_deferring_client]
3     create_$type blist "a b $large c d"
4
5     bpop_command $rd $pop blist 1
6     assert_equal {blist a} [$rd read]
7     if {$pop == "BLPOP"} {
8         bpop_command $rd BRPOP blist 1
9     } else {
10        bpop_command $rd BLMPOP_RIGHT blist 1
11    }
12    assert_equal {blist d} [$rd read]
13
14    bpop_command $rd $pop blist 1
15    assert_equal {blist b} [$rd read]
16    if {$pop == "BLPOP"} {
17        bpop_command $rd BRPOP blist 1
18    } else {
19        bpop_command $rd BLMPOP_RIGHT blist 1

```

```

20     }
21     assert_equal {blist c} [$rd read]
22
23     assert_equal 1 [r llen blist]
24     $rd close
25 }

```

功能说明:

- `BLPOP`: 阻塞式左侧弹出
- 如果列表为空，客户端会阻塞等待
- 如果列表有元素，立即返回

执行结果:

```

1 [ok]: BLPOP: single existing list - quicklist (1 ms)

```

5.10 测试用例 #10: SDIFF fuzzing

测试文件: `tests/unit/type/set.tcl:447+`

测试目的: 模糊测试集合差集操作

测试代码:

```

1 test "SDIFF fuzzing" {
2     for {set j 0} {$j < 100} {incr j} {
3         unset -nocomplain s
4         array set s {}
5         set args {}
6         set num_sets [expr {[randomInt 10]+1}]
7         for {set i 0} {$i < $num_sets} {incr i} {
8             set num_elements [randomInt 100]
9             r del set_${i}{t}
10            lappend args set_${i}{t}
11            while {$num_elements} {
12                set ele [randomValue]
13                r sadd set_${i}{t} $ele
14                if {$i == 0} {
15                    set s($ele) x
16                } else {
17                    unset -nocomplain s($ele)
18                }
19                incr num_elements -1
20            }
21        }
22        set result [lsort [r sdiff {*}$args]]
23        assert_equal $result [lsort [array names s]]
24    }
25 }

```

功能说明:

- 使用随机数据进行压力测试
- `SDIFF key1 key2 ...`: 返回第一个集合与其他集合的差集
- 验证边界情况和异常输入的处理

执行结果:

```
1 [ok]: SDIFF fuzzing (1284 ms)
```

6 实验总结

通过该作业，我很好理解了 redis 的功能以及测试框架架构，理解了 Redis 测试框架的工作机制：

```
1 tests/
2 └─ test_helper.tcl      # 测试框架核心
3 └─ unit/                # 单元测试
4   │   └─ type/          # 数据类型测试
5   │   │   └─ hash.tcl   # 哈希测试
6   │   │   └─ set.tcl    # 集合测试
7   │   │   └─ list.tcl   # 列表测试
8   │   │   └─ string.tcl # 字符串测试
9   │   │   └─ zset.tcl   # 有序集合测试
10  │   └─ acl.tcl         # 访问控制测试
11  └─ ...
12 └─ integration/        # 集成测试
13 └─ cluster/            # 集群测试
```

测试用例编写按照以下模式：

```
1 # 1. 启动测试服务器
2 start_server {tags {"数据类型"}} {
3
4     # 2. 配置测试参数
5     r config set 参数名 值
6
7     # 3. 定义测试用例
8     test {测试描述} {
9         # 测试步骤
10        r 命令 参数...
11
12        # 断言验证
13        assert_equal 期望值 实际值
14    } {期望返回值}
15 }
```

可扩展性强，且覆盖正常流程和异常场景，使用随机数据增强测试鲁棒性，每个功能都有对应的单元测试。对我之后程序开发过程的测试阶段有着极大的帮助。

7 附录A：完整测试运行结果

```
1 Cleanup: may take some time... OK
2 Starting test server at port 21080
3 [ready]: 6960
4 Testing unit/type/list
5 [ready]: 6958
6 Testing unit/type/hash
7 [ready]: 6959
8 Testing unit/type/set
9 [ready]: 6961
10 [ready]: 6965
11 [ready]: 6963
12 [ready]: 6964
13 [ready]: 6967
14 [ready]: 6966
15 [ready]: 6962
16 [ready]: 6968
17 [ready]: 6969
18 [ready]: 6970
19 [ready]: 6972
20 [ready]: 6971
21 [ready]: 6973
22 [ok]: HSET/HLEN - Small hash creation (1 ms)
23 [ok]: Is the small hash encoded with a listpack? (0 ms)
24 [ok]: SADD, SCARD, SISEMEMBER, SMISEMEMBER, SISEMEMBERS basics - listpack (1 ms)
25 [ok]: packed node check compression with insert and pop (1 ms)
26 [ok]: packed node check compression combined with trim (1 ms)
27 [ok]: packed node check compression with lset (1 ms)
28 [ok]: plain node check compression with insert and pop (2 ms)
29 [ok]: plain node check compression combined with trim (1 ms)
30 [ok]: plain node check compression with lset (1 ms)
31 [ok]: HRANDFIELD - listpack (7 ms)
32 [ok]: SADD, SCARD, SISEMEMBER, SMISEMEMBER, SISEMEMBERS basics - hashtable (9 ms)
33 [ok]: SADD, SCARD, SISEMEMBER, SMISEMEMBER, SISEMEMBERS basics - intset (1 ms)
34 [ok]: SMISEMEMBER SISEMEMBERS SCARD against non set (0 ms)
35 [ok]: SMISEMEMBER SISEMEMBERS SCARD against non existing key (1 ms)
36 [ok]: SMISEMEMBER requires one or more members (0 ms)
37 [ok]: SADD against non set (0 ms)
38 [ok]: SADD a non-integer against a small intset (0 ms)
39 [ok]: HRANDFIELD - hashtable (5 ms)
40 [ok]: HRANDFIELD with RESP3 (1 ms)
41 [ok]: HRANDFIELD count of 0 is handled correctly (0 ms)
42 [ok]: HRANDFIELD count overflow (0 ms)
43 [ok]: HRANDFIELD with <count> against non existing key (0 ms)
44 [ok]: HRANDFIELD count of 0 is handled correctly - emptyarray (0 ms)
45 [ok]: HRANDFIELD with <count> against non existing key - emptyarray (0 ms)
46 [ok]: SADD a non-integer against a large intset (7 ms)
47 [ok]: SADD an integer larger than 64 bits (0 ms)
48 [ok]: SADD an integer larger than 64 bits to a large intset (6 ms)
```

49 [ok]: SADD overflows the maximum allowed integers in an intset - single (13 ms)
50 [ok]: SADD overflows the maximum allowed elements in a listpack - single (4 ms)
51 [ok]: SADD overflows the maximum allowed integers in an intset - multiple (0 ms)
52 [ok]: SADD overflows the maximum allowed elements in a listpack - multiple (0 ms)
53 [ok]: SADD overflows the maximum allowed integers in an intset - single_multiple (0 ms)
54 [ok]: SADD overflows the maximum allowed elements in a listpack - single_multiple (0 ms)
55 [ok]: Variadic SADD (0 ms)
56 [ok]: HRANDFIELD with <count> - hashtable (46 ms)
57 [ok]: Set encoding after DEBUG RELOAD (49 ms)
58 [ok]: SREM basics - \$type (1 ms)
59 [ok]: SREM basics - \$type (3 ms)
60 [ok]: SREM basics - intset (0 ms)
61 [ok]: SREM with multiple arguments (1 ms)
62 [ok]: SREM variadic version with more args needed to destroy the key (0 ms)
63 [ok]: HRANDFIELD with <count> - listpack (37 ms)
64 [ok]: SINTERCARD with illegal arguments (0 ms)
65 [ok]: SINTERCARD against non-set should throw error (1 ms)
66 [ok]: SINTERCARD against non-existing key (0 ms)
67 [ok]: Generated sets must be encoded correctly - regular (0 ms)
68 [ok]: SINTER with two sets - regular (0 ms)
69 [ok]: SINTERCARD with two sets - regular (0 ms)
70 [ok]: SINTERSTORE with two sets - regular (0 ms)
71 [ok]: SINTERSTORE with two sets, after a DEBUG RELOAD - regular (5 ms)
72 [ok]: SUNION with two sets - regular (2 ms)
73 [ok]: SUNIONSTORE with two sets - regular (2 ms)
74 [ok]: SINTER against three sets - regular (0 ms)
75 [ok]: SINTERCARD against three sets - regular (0 ms)
76 [ok]: SINTERSTORE with three sets - regular (0 ms)
77 [ok]: SUNION with non existing keys - regular (2 ms)
78 [ok]: SDIFF with two sets - regular (1 ms)
79 [ok]: SDIFF with three sets - regular (0 ms)
80 [ok]: SDIFFSTORE with three sets - regular (0 ms)
81 [ok]: SINTER/SUNION/SDIFF with three same sets - regular (2 ms)
82 [ok]: HSET/HLEN - Big hash creation (35 ms)
83 [ok]: Is the big hash encoded with an hash table? (0 ms)
84 [ok]: HGET against the small hash (1 ms)
85 [ok]: Generated sets must be encoded correctly - intset (0 ms)
86 [ok]: SINTER with two sets - intset (0 ms)
87 [ok]: SINTERCARD with two sets - intset (0 ms)
88 [ok]: SINTERSTORE with two sets - intset (1 ms)
89 [ok]: SINTERSTORE with two sets, after a DEBUG RELOAD - intset (3 ms)
90 [ok]: SUNION with two sets - intset (2 ms)
91 [ok]: SUNIONSTORE with two sets - intset (2 ms)
92 [ok]: SINTER against three sets - intset (0 ms)
93 [ok]: SINTERCARD against three sets - intset (1 ms)
94 [ok]: SINTERSTORE with three sets - intset (0 ms)

95 [ok]: SUNION with non existing keys - intset (2 ms)
96 [ok]: SDIFF with two sets - intset (0 ms)
97 [ok]: SDIFF with three sets - intset (0 ms)
98 [ok]: SDIFFSTORE with three sets - intset (0 ms)
99 [ok]: SINTER/SUNION/SDIFF with three same sets - intset (2 ms)
100 [ok]: SINTERSTORE with two listpack sets where result is intset (0 ms)
101 [ok]: HGET against the big hash (27 ms)
102 [ok]: HGET against non existing key (0 ms)
103 [ok]: HSET in update and insert mode (0 ms)
104 [ok]: HSETNX target key missing - small hash (0 ms)
105 [ok]: HSETNX target key exists - small hash (0 ms)
106 [ok]: HSETNX target key missing - big hash (1 ms)
107 [ok]: HSETNX target key exists - big hash (0 ms)
108 [ok]: HSET/HMSET wrong number of args (0 ms)
109 [ok]: HMSET - small hash (0 ms)
110 [ok]: HMSET - big hash (4 ms)
111 [ok]: HMGET against non existing key and fields (0 ms)
112 [ok]: SINTERSTORE with two hashtable sets where result is intset (7 ms)
113 [ok]: Hash commands against wrong type (1 ms)
114 [ok]: HMGET - small hash (0 ms)
115 [ok]: SUNION hashtable and listpack (1 ms)
116 [ok]: SDIFF with first set empty (0 ms)
117 [ok]: SDIFF with same set two times (0 ms)
118 [ok]: HMGET - big hash (4 ms)
119 [ok]: HKEYS - small hash (0 ms)
120 [ok]: HKEYS - big hash (3 ms)
121 [ok]: HVALS - small hash (0 ms)
122 [ok]: HVALS - big hash (0 ms)
123 [ok]: HGETALL - small hash (0 ms)
124 [ok]: HGETALL - big hash (6 ms)
125 [ok]: HGETALL against non-existing key (0 ms)
126 [ok]: HDEL and return value (0 ms)
127 [ok]: HDEL - more than a single value (0 ms)
128 [ok]: HDEL - hash becomes empty before deleting all specified fields (0 ms)
129 [ok]: HEXISTS (1 ms)
130 [ok]: Is a ziplist encoded Hash promoted on big payload? (0 ms)
131 [ok]: HINCRBY against non existing database key (0 ms)
132 [ok]: HINCRBY HINCRBYFLOAT against non-integer increment value (1 ms)
133 [ok]: HINCRBY against non existing hash key (0 ms)
134 [ok]: HINCRBY against hash key created by hincrby itself (0 ms)
135 [ok]: HINCRBY against hash key originally set with HSET (0 ms)
136 [ok]: HINCRBY over 32bit value (0 ms)
137 [ok]: HINCRBY over 32bit value with over 32bit increment (0 ms)
138 [ok]: HINCRBY fails against hash value with spaces (left) (1 ms)
139 [ok]: HINCRBY fails against hash value with spaces (right) (0 ms)
140 [ok]: HINCRBY can detect overflows (0 ms)
141 [ok]: HINCRBYFLOAT against non existing database key (0 ms)
142 [ok]: HINCRBYFLOAT against non existing hash key (0 ms)
143 [ok]: HINCRBYFLOAT against hash key created by hincrby itself (1 ms)
144 [ok]: HINCRBYFLOAT against hash key originally set with HSET (0 ms)
145 [ok]: HINCRBYFLOAT over 32bit value (0 ms)
146 [ok]: HINCRBYFLOAT over 32bit value with over 32bit increment (0 ms)

```
147 [ok]: HINCRBYFLOAT fails against hash value with spaces (left) (0 ms)
148 [ok]: HINCRBYFLOAT fails against hash value with spaces (right) (1 ms)
149 [ok]: HINCRBYFLOAT fails against hash value that contains a null-terminator in
150 the middle (0 ms)
    )
        [ok]: HSTRLEN against the small hash (0 ms)
151 [ok]: HSTRLEN against the big hash (28 ms)
152 [ok]: HSTRLEN against non existing field (0 ms)
153 [ok]: HSTRLEN corner cases (1 ms)
154 [ok]: HINCRBYFLOAT over hash-max-listpack-value encoded with a listpack (1 ms)
155 [ok]: HGETDEL input validation (1 ms)
156 [ok]: HGETDEL basic test (1 ms)
157 [ok]: HGETDEL test with non existing fields (0 ms)
158 [ok]: HGETDEL basic test (1 ms)
159 [ok]: HGETDEL test with non existing fields (0 ms)
160 [ok]: HGETDEL propagated as HDEL command to replica (43 ms)
161 [ok]: Hash ziplist regression test for large keys (0 ms)
162 [ok]: Hash fuzzing #1 - 10 fields (9 ms)
163 [ok]: Hash fuzzing #2 - 10 fields (5 ms)
164 [ok]: Hash fuzzing #1 - 512 fields (392 ms)
165 [ok]: Hash fuzzing #2 - 512 fields (324 ms)
166 [ok]: Check for memory leaks (pid 6983) (1189 ms)
167 [ok]: Stress test the hash ziplist -> hashtable encoding conversion (374 ms)
168 [ok]: Hash ziplist of various encodings (2 ms)
169 [ok]: Hash ziplist of various encodings - sanitize dump (1 ms)
170 [ok]: HINCRBYFLOAT does not allow NaN or Infinity (0 ms)
171 [ok]: SDIFF fuzzing (1284 ms)
172 [ok]: SDIFF against non-set should throw error (0 ms)
173 [ok]: SDIFF should handle non existing key as empty (0 ms)
174 [ok]: SDIFFSTORE against non-set should throw error (1 ms)
175 [ok]: SDIFFSTORE should handle non existing key as empty (0 ms)
176 [ok]: SINTER against non-set should throw error (0 ms)
177 [ok]: SINTER should handle non existing key as empty (1 ms)
178 [ok]: SINTER with same integer elements but different encoding (0 ms)
179 [ok]: SINTERSTORE against non-set should throw error (0 ms)
180 [ok]: SINTERSTORE against non existing keys should delete dstkey (0 ms)
181 [ok]: Test LPUSH and LPOP on plain nodes (5 ms)
182 [ok]: SUNION against non-set should throw error (0 ms)
183 [ok]: SUNION should handle non existing key as empty (0 ms)
184 [ok]: Test LINDEX and LINSERT on plain nodes (1 ms)
185 [ok]: SUNIONSTORE against non-set should throw error (1 ms)
186 [ok]: Test LTRIM on plain nodes (0 ms)
187 [ok]: Test LREM on plain nodes (0 ms)
188 [ok]: SUNIONSTORE should handle non existing key as empty (0 ms)
189 [ok]: SUNIONSTORE against non existing keys should delete dstkey (1 ms)
190 [ok]: Test LPOS on plain nodes (1 ms)
191 [ok]: SPOP basics - listpack (0 ms)
192 [ok]: SPOP with <count>=1 - listpack (0 ms)
193 [ok]: Test LMOVE on plain nodes (0 ms)
194 [ok]: Test LSET with packed / plain combinations (2 ms)
195 [ok]: Test LSET with packed is split in the middle (0 ms)
196 [ok]: SRANDMEMBER - listpack (3 ms)
```

```
197 [ok]: SPOP basics - intset (0 ms)
198 [ok]: SPOP with <count>=1 - intset (0 ms)
199 [ok]: Test LSET with packed consist only one item (2 ms)
200 [ok]: Crash due to delete entry from a compress quicklist node (0 ms)
201 [ok]: Crash due to split quicklist node wrongly (0 ms)
202 [ok]: SRANDMEMBER - intset (3 ms)
203 [ok]: Test LPUSH and LPOP on plain nodes (5 ms)
204 [ok]: SPOP integer from listpack set (1 ms)
205 [ok]: Test LINDEX and LINSERT on plain nodes (0 ms)
206 [ok]: Test LTRIM on plain nodes (0 ms)
207 [ok]: Test LREM on plain nodes (0 ms)
208 [ok]: SPOP with <count> - listpack (1 ms)
209 [ok]: Test LPOS on plain nodes (0 ms)
210 [ok]: Test LMOVE on plain nodes (1 ms)
211 [ok]: SPOP with <count> - intset (1 ms)
212 [ok]: SPOP with <count> - hashtable (1 ms)
213 [ok]: Test LSET with packed / plain combinations (1 ms)
214 [ok]: Test LSET with packed is split in the middle (1 ms)
215 [ok]: SPOP using integers, testing Knuth's and Floyd's algorithm (1 ms)
216 [ok]: SPOP using integers with Knuth's algorithm (0 ms)
217 [ok]: SPOP new implementation: code path #1 intset (1 ms)
218 [ok]: Test LSET with packed consist only one item (1 ms)
219 [ok]: Crash due to delete entry from a compress quicklist node (1 ms)
220 [ok]: SPOP new implementation: code path #2 intset (1 ms)
221 [ok]: Crash due to split quicklist node wrongly (0 ms)
222 [ok]: SPOP new implementation: code path #3 intset (0 ms)
223 [ok]: SPOP new implementation: code path #1 listpack (1 ms)
224 [ok]: SPOP new implementation: code path #2 listpack (1 ms)
225 [ok]: SPOP new implementation: code path #3 listpack (0 ms)
226 [ok]: SPOP new implementation: code path #1 propagate as DEL or UNLINK (90 ms)
227 [ok]: SRANDMEMBER count of 0 is handled correctly (0 ms)
228 [ok]: SRANDMEMBER with <count> against non existing key (0 ms)
229 [ok]: SRANDMEMBER count overflow (0 ms)
230 [ok]: SRANDMEMBER count of 0 is handled correctly - emptyarray (0 ms)
231 [ok]: SRANDMEMBER with <count> against non existing key - emptyarray (0 ms)
232 [ok]: SRANDMEMBER with <count> - listpack (6 ms)
233 [ok]: SRANDMEMBER with <count> - intset (6 ms)
234 [ok]: SRANDMEMBER with <count> - hashtable (7 ms)
235 [ok]: SRANDMEMBER histogram distribution - listpack (22 ms)
236 [ok]: SRANDMEMBER histogram distribution - intset (23 ms)
237 [ok]: SRANDMEMBER histogram distribution - hashtable (24 ms)
238 [ok]: Check for memory leaks (pid 6984) (597 ms)
239 [ok]: Check for memory leaks (pid 7078) (610 ms)
240 [1/3 done]: unit/type/hash (3 seconds)
241 [ok]: LPOS basic usage - quicklist (0 ms)
242 [ok]: LPOS RANK (positive, negative and zero rank) option (1 ms)
243 [ok]: LPOS COUNT option (0 ms)
244 [ok]: LPOS COUNT + RANK option (0 ms)
245 [ok]: LPOS non existing key (0 ms)
246 [ok]: LPOS no match (0 ms)
247 [ok]: LPOS MAXLEN (0 ms)
248 [ok]: LPOS when RANK is greater than matches (1 ms)
```


249 [ok]: LPUSH, RPUSH, LLENGTH, LINDEX, LPOP - quicklist (1 ms)
250 [ok]: LPOP/RPOP with wrong number of arguments (0 ms)
251 [ok]: RPOP/LPOP with the optional count argument - quicklist (0 ms)
252 [ok]: LPOS basic usage - listpack (0 ms)
253 [ok]: LPOS RANK (positive, negative and zero rank) option (0 ms)
254 [ok]: LPOS COUNT option (0 ms)
255 [ok]: LPOS COUNT + RANK option (0 ms)
256 [ok]: LPOS non existing key (0 ms)
257 [ok]: LPOS no match (0 ms)
258 [ok]: LPOS MAXLEN (0 ms)
259 [ok]: LPOS when RANK is greater than matches (0 ms)
260 [ok]: LPUSH, RPUSH, LLENGTH, LINDEX, LPOP - listpack (1 ms)
261 [ok]: LPOP/RPOP with wrong number of arguments (0 ms)
262 [ok]: RPOP/LPOP with the optional count argument - listpack (1 ms)
263 [ok]: LPOP/RPOP with the count 0 returns an empty array in RESP3 (0 ms)
264 [ok]: LPOP/RPOP against non existing key in RESP3 (0 ms)
265 [ok]: LPOP/RPOP with <count> against non existing key in RESP3 (0 ms)
266 [ok]: LPOP/RPOP with the count 0 returns an empty array in RESP2 (1 ms)
267 [ok]: LPOP/RPOP against non existing key in RESP2 (0 ms)
268 [ok]: LPOP/RPOP with <count> against non existing key in RESP2 (0 ms)
269 [ok]: Variadic RPUSH/LPUSH (0 ms)
270 [ok]: DEL a list (0 ms)
271 [ok]: BLPOP: single existing list - quicklist (1 ms)
272 [ok]: BLPOP: multiple existing lists - quicklist (1 ms)
273 [ok]: BLPOP: second list has an entry - quicklist (0 ms)
274 [ok]: BLMPOP_LEFT: single existing list - quicklist (1 ms)
275 [ok]: BLMPOP_LEFT: multiple existing lists - quicklist (1 ms)
276 [ok]: BLMPOP_LEFT: second list has an entry - quicklist (0 ms)
277 [ok]: BRPOPLPUSH - quicklist (1 ms)
278 [ok]: BLMOVE left left - quicklist (0 ms)
279 [ok]: BLMOVE left right - quicklist (1 ms)
280 [ok]: BLMOVE right left - quicklist (0 ms)
281 [ok]: BLMOVE right right - quicklist (1 ms)
282 [ok]: BLPOP: single existing list - listpack (1 ms)
283 [ok]: BLPOP: multiple existing lists - listpack (1 ms)
284 [ok]: BLPOP: second list has an entry - listpack (1 ms)
285 [ok]: BLMPOP_LEFT: single existing list - listpack (1 ms)
286 [ok]: BLMPOP_LEFT: multiple existing lists - listpack (1 ms)
287 [ok]: BLMPOP_LEFT: second list has an entry - listpack (1 ms)
288 [ok]: BRPOPLPUSH - listpack (0 ms)
289 [ok]: BLMOVE left left - listpack (1 ms)
290 [ok]: BLMOVE left right - listpack (1 ms)
291 [ok]: BLMOVE right left - listpack (0 ms)
292 [ok]: BLMOVE right right - listpack (1 ms)
293 [ok]: BLPOP, LPUSH + DEL should not awake blocked client (0 ms)
294 [ok]: BLPOP, LPUSH + DEL + SET should not awake blocked client (1 ms)
295 [ok]: BLMPOP_LEFT, LPUSH + DEL should not awake blocked client (0 ms)
296 [ok]: BLMPOP_LEFT, LPUSH + DEL + SET should not awake blocked client (1 ms)
297 [ok]: BLPOP with same key multiple times should work (issue #801) (0 ms)
298 [ok]: MULTI/EXEC is isolated from the point of view of BLPOP (1 ms)
299 [ok]: BLPOP with variadic LPUSH (0 ms)
300 [ok]: MULTI/EXEC is isolated from the point of view of BLMPOP_LEFT (1 ms)

```
301 [ok]: BLMPOP_LEFT with variadic LPUSH (0 ms)
302 [ok]: BRPOPLPUSH with zero timeout should block indefinitely (1 ms)
303 [ok]: BLMOVE left left with zero timeout should block indefinitely (0 ms)
304 [ok]: BLMOVE left right with zero timeout should block indefinitely (0 ms)
305 [ok]: BLMOVE right left with zero timeout should block indefinitely (1 ms)
306 [ok]: BLMOVE right right with zero timeout should block indefinitely (0 ms)
307 [ok]: BLMOVE (left, left) with a client BLPOPing the target list (1 ms)
308 [ok]: BLMOVE (left, right) with a client BLPOPing the target list (0 ms)
309 [ok]: BLMOVE (right, left) with a client BLPOPing the target list (0 ms)
310 [ok]: BLMOVE (right, right) with a client BLPOPing the target list (0 ms)
311 [ok]: BRPOPLPUSH with wrong source type (0 ms)
312 [ok]: BRPOPLPUSH with wrong destination type (1 ms)
313 [ok]: BRPOPLPUSH maintains order of elements after failure (0 ms)
314 [ok]: BRPOPLPUSH with multiple blocked clients (1 ms)
315 [ok]: BLMPOP with multiple blocked clients (1 ms)
316 [ok]: Linked LMOVEs (0 ms)
317 [ok]: Circular BRPOPLPUSH (1 ms)
318 [ok]: Self-referential BRPOPLPUSH (0 ms)
319 [ok]: BRPOPLPUSH inside a transaction (0 ms)
320 [ok]: PUSH resulting from BRPOPLPUSH affect WATCH (1 ms)
321 [ok]: BRPOPLPUSH does not affect WATCH while still blocked (0 ms)
322 [ok]: BRPOPLPUSH timeout (1016 ms)
323 [ok]: SWAPDB awakes blocked client (0 ms)
324 [ok]: SWAPDB wants to wake blocked client, but the key already expired (136
ms)
325 [ok]: MULTI + LPUSH + EXPIRE + DEBUG SLEEP on blocked client, key already
expired (301 ms)
326 [ok]: BLPOP unblock but the key is expired and then block again - reprocessing
command (1008 ms
327 )
[ok]: BLPOP when new key is moved into place (0 ms)
328 [ok]: BLPOP when result key is created by SORT..STORE (1 ms)
329 [ok]: BLMPOP_LEFT when new key is moved into place (0 ms)
330 [ok]: BLMPOP_LEFT when result key is created by SORT..STORE (1 ms)
331 [ok]: BLPOP: timeout value out of range (0 ms)
332 [ok]: BLPOP: with single empty list argument (1 ms)
333 [ok]: BLPOP: with negative timeout (0 ms)
334 [ok]: BLPOP: with non-integer timeout (0 ms)
335 [ok]: BLPOP: with zero timeout should block indefinitely (0 ms)
336 [ok]: BLPOP: with 0.001 timeout should not block indefinitely (23 ms)
337 [ok]: BLPOP: second argument is not a list (0 ms)
338 [ok]: SRANDMEMBER with a dict containing long chain (3537 ms)
339 [ok]: SMOVE basics - from regular set to intset (0 ms)
340 [ok]: SMOVE basics - from intset to regular set (0 ms)
341 [ok]: SMOVE non existing key (0 ms)
342 [ok]: SMOVE non existing src set (1 ms)
343 [ok]: SMOVE from regular set to non existing destination set (0 ms)
344 [ok]: SMOVE from intset to non existing destination set (1 ms)
345 [ok]: SMOVE wrong src key type (0 ms)
346 [ok]: SMOVE wrong dst key type (0 ms)
347 [ok]: SMOVE with identical source and destination (0 ms)
348 [ok]: SMOVE only notify dstset when the addition is successful (1 ms)
```

349 [ok]: intsets implementation stress testing (702 ms)
350 [ok]: BLPOP: timeout (1083 ms)
351 [ok]: BLPOP: arguments are empty (1 ms)
352 [ok]: BRPOP: with single empty list argument (0 ms)
353 [ok]: BRPOP: with negative timeout (0 ms)
354 [ok]: BRPOP: with non-integer timeout (0 ms)
355 [ok]: BRPOP: with zero timeout should block indefinitely (1 ms)
356 [ok]: BRPOP: with 0.001 timeout should not block indefinitely (23 ms)
357 [ok]: BRPOP: second argument is not a list (0 ms)
358 [ok]: Check for memory leaks (pid 6985) (554 ms)
359 [2/3 done]: unit/type/set (7 seconds)
360 [ok]: BRPOP: timeout (1087 ms)
361 [ok]: BRPOP: arguments are empty (1 ms)
362 [ok]: BLMPOP_LEFT: with single empty list argument (1 ms)
363 [ok]: BLMPOP_LEFT: with negative timeout (1 ms)
364 [ok]: BLMPOP_LEFT: with non-integer timeout (0 ms)
365 [ok]: BLMPOP_LEFT: with zero timeout should block indefinitely (1 ms)
366 [ok]: BLMPOP_LEFT: with 0.001 timeout should not block indefinitely (24 ms)
367 [ok]: BLMPOP_LEFT: second argument is not a list (0 ms)
368 [ok]: BLMPOP_LEFT: timeout (1084 ms)
369 [ok]: BLMPOP_LEFT: arguments are empty (4 ms)
370 [ok]: BLMPOP_RIGHT: with single empty list argument (2 ms)
371 [ok]: BLMPOP_RIGHT: with negative timeout (0 ms)
372 [ok]: BLMPOP_RIGHT: with non-integer timeout (1 ms)
373 [ok]: BLMPOP_RIGHT: with zero timeout should block indefinitely (2 ms)
374 [ok]: BLMPOP_RIGHT: with 0.001 timeout should not block indefinitely (24 ms)
375 [ok]: BLMPOP_RIGHT: second argument is not a list (1 ms)
376 [ok]: BLMPOP_RIGHT: timeout (1070 ms)
377 [ok]: BLMPOP_RIGHT: arguments are empty (2 ms)
378 [ok]: BLPOP inside a transaction (0 ms)
379 [ok]: BLMPOP_LEFT inside a transaction (1 ms)
380 [ok]: BLMPOP propagate as pop with count command to replica (199 ms)
381 [ok]: LPUSHX, RPUSHX - generic (1 ms)
382 [ok]: LPUSHX, RPUSHX - quicklist (1 ms)
383 [ok]: LINSERT - quicklist (2 ms)
384 [ok]: LPUSHX, RPUSHX - listpack (0 ms)
385 [ok]: LINSERT - listpack (2ms)
386 [ok]: LINSERT raise error on bad syntax (0 ms)
387 [ok]: LINSERT against non-list value error (0 ms)
388 [ok]: LINSERT against non existing key (0 ms)
389 [ok]: LINDEX consistency test - listpack (36 ms)
390 [ok]: LINDEX random access - listpack (15 ms)
391 [ok]: Check if list is still ok after a DEBUG RELOAD - listpack (29 ms)
392 [ok]: LINDEX consistency test - listpack (39 ms)
393 [ok]: LINDEX random access - listpack (27 ms)
394 [ok]: Check if list is still ok after a DEBUG RELOAD - listpack (62 ms)
395 [ok]: LINDEX consistency test - quicklist (20 ms)
396 [ok]: LINDEX random access - quicklist (14 ms)
397 [ok]: Check if list is still ok after a DEBUG RELOAD - quicklist (29 ms)
398 [ok]: LINDEX consistency test - quicklist (40 ms)
399 [ok]: LINDEX random access - quicklist (26 ms)
400 [ok]: Check if list is still ok after a DEBUG RELOAD - quicklist (54 ms)

401 [ok]: LLEN against non-list value error (0 ms)
402 [ok]: LLEN against non existing key (0 ms)
403 [ok]: LINDEX against non-list value error (0 ms)
404 [ok]: LINDEX against non existing key (0 ms)
405 [ok]: LPUSH against non-list value error (0 ms)
406 [ok]: RPUSH against non-list value error (0 ms)
407 [ok]: RPOPLPUSH base case - quicklist (1 ms)
408 [ok]: LMOVE left left base case - quicklist (0 ms)
409 [ok]: LMOVE left right base case - quicklist (1 ms)
410 [ok]: LMOVE right left base case - quicklist (0 ms)
411 [ok]: LMOVE right right base case - quicklist (1 ms)
412 [ok]: RPOPLPUSH with the same list as src and dst - quicklist (0 ms)
413 [ok]: LMOVE left left with the same list as src and dst - quicklist (0 ms)
414 [ok]: LMOVE left right with the same list as src and dst - quicklist (1 ms)
415 [ok]: LMOVE right left with the same list as src and dst - quicklist (0 ms)
416 [ok]: LMOVE right right with the same list as src and dst - quicklist (0 ms)
417 [ok]: RPOPLPUSH with quicklist source and existing target quicklist (1 ms)
418 [ok]: LMOVE left left with quicklist source and existing target quicklist (0 ms)
419 [ok]: LMOVE left right with quicklist source and existing target quicklist (1 ms)
420 [ok]: LMOVE right left with quicklist source and existing target quicklist (0 ms)
421 [ok]: LMOVE right right with quicklist source and existing target quicklist (1 ms)
422 [ok]: RPOPLPUSH with quicklist source and existing target listpack (0 ms)
423 [ok]: LMOVE left left with quicklist source and existing target listpack (1 ms)
424 [ok]: LMOVE left right with quicklist source and existing target listpack (0 ms)
425 [ok]: LMOVE right left with quicklist source and existing target listpack (1 ms)
426 [ok]: LMOVE right right with quicklist source and existing target listpack (0 ms)
427 [ok]: RPOPLPUSH base case - listpack (1 ms)
428 [ok]: LMOVE left left base case - listpack (0 ms)
429 [ok]: LMOVE left right base case - listpack (1 ms)
430 [ok]: LMOVE right left base case - listpack (0 ms)
431 [ok]: LMOVE right right base case - listpack (1 ms)
432 [ok]: RPOPLPUSH with the same list as src and dst - listpack (0 ms)
433 [ok]: LMOVE left left with the same list as src and dst - listpack (0 ms)
434 [ok]: LMOVE left right with the same list as src and dst - listpack (0 ms)
435 [ok]: LMOVE right left with the same list as src and dst - listpack (0 ms)
436 [ok]: LMOVE right right with the same list as src and dst - listpack (0 ms)
437 [ok]: RPOPLPUSH with listpack source and existing target quicklist (0 ms)
438 [ok]: LMOVE left left with listpack source and existing target quicklist (1 ms)
439 [ok]: LMOVE left right with listpack source and existing target quicklist (0 ms)
440 [ok]: LMOVE right left with listpack source and existing target quicklist (1 ms)

441 [ok]: LMOVE right right with listpack source and existing target quicklist (0 ms)
442 [ok]: RPOPLPUSH with listpack source and existing target listpack (1 ms)
443 [ok]: LMOVE left left with listpack source and existing target listpack (0 ms)
444 [ok]: LMOVE left right with listpack source and existing target listpack (0 ms)
445 [ok]: LMOVE right left with listpack source and existing target listpack (1 ms)
446 [ok]: LMOVE right right with listpack source and existing target listpack (0 ms)
447 [ok]: RPOPLPUSH against non existing key (0 ms)
448 [ok]: RPOPLPUSH against non list src key (0 ms)
449 [ok]: RPOPLPUSH against non list dst key - quicklist (1 ms)
450 [ok]: RPOPLPUSH against non list dst key - listpack (0 ms)
451 [ok]: RPOPLPUSH against non existing src key (0 ms)
452 [ok]: Basic LPOP/RPOP/LMPOP - quicklist (1 ms)
453 [ok]: Basic LPOP/RPOP/LMPOP - listpack (0 ms)
454 [ok]: LPOP/RPOP/LMPOP against empty list (1 ms)
455 [ok]: LPOP/RPOP/LMPOP NON-BLOCK or BLOCK against non list value (0 ms)
456 [ok]: Mass RPOP/LPOP - listpack (13 ms)
457 [ok]: Mass RPOP/LPOP - listpack (26 ms)
458 [ok]: LMPOP with illegal argument (1 ms)
459 [ok]: LMPOP single existing list - quicklist (1 ms)
460 [ok]: LMPOP multiple existing lists - quicklist (1 ms)
461 [ok]: LMPOP single existing list - listpack (1 ms)
462 [ok]: LMPOP multiple existing lists - listpack (1 ms)
463 [ok]: LMPOP propagate as pop with count command to replica (35 ms)
464 [ok]: LRANGE basics - quicklist (1 ms)
465 [ok]: LRANGE inverted indexes - quicklist (0 ms)
466 [ok]: LRANGE out of range indexes including the full list - quicklist (0 ms)
467 [ok]: LRANGE out of range negative end index - quicklist (1 ms)
468 [ok]: LRANGE basics - listpack (0 ms)
469 [ok]: LRANGE inverted indexes - listpack (0 ms)
470 [ok]: LRANGE out of range indexes including the full list - listpack (1 ms)
471 [ok]: LRANGE out of range negative end index - listpack (0 ms)
472 [ok]: LRANGE against non existing key (0 ms)
473 [ok]: LRANGE with start > end yields an empty array for backward compatibility (0 ms)
474 [ok]: LTRIM basics - quicklist (4 ms)
475 [ok]: LTRIM out of range negative end index - quicklist (0 ms)
476 [ok]: LSET - quicklist (1 ms)
477 [ok]: LSET out of range index - quicklist (0 ms)
478 [ok]: LTRIM basics - listpack (3 ms)
479 [ok]: LTRIM out of range negative end index - listpack (1 ms)
480 [ok]: LSET - listpack (0 ms)
481 [ok]: LSET out of range index - listpack (0 ms)
482 [ok]: LSET against non existing key (0 ms)
483 [ok]: LSET against non list value (0 ms)
484 [ok]: LREM remove all the occurrences - quicklist (1 ms)
485 [ok]: LREM remove the first occurrence - quicklist (0 ms)
486 [ok]: LREM remove non existing element - quicklist (0 ms)
487 [ok]: LREM starting from tail with negative count - quicklist (0 ms)

```
488 [ok]: LREM starting from tail with negative count (2) - quicklist (1 ms)
489 [ok]: LREM deleting objects that may be int encoded - quicklist (0 ms)
490 [ok]: LREM remove all the occurrences - listpack (0 ms)
491 [ok]: LREM remove the first occurrence - listpack (0 ms)
492 [ok]: LREM remove non existing element - listpack (0 ms)
493 [ok]: LREM starting from tail with negative count - listpack (0 ms)
494 [ok]: LREM starting from tail with negative count (2) - listpack (0 ms)
495 [ok]: LREM deleting objects that may be int encoded - listpack (0 ms)
496 [ok]: Regression for bug 593 - chaining BRPOPLPUSH with other blocking cmds (1
ms)
497 [ok]: BLPOP/BLMOVE should increase dirty (1 ms)
498 [ok]: client unblock tests (2 ms)
499 [ok]: client unblock tests (1 ms)
500 [ok]: List listpack -> quicklist encoding conversion (1 ms)
501 [ok]: List listpack -> quicklist encoding conversion (1 ms)
502 [ok]: List quicklist -> listpack encoding conversion (2 ms)
503 [ok]: List encoding conversion when RDB loading (8 ms)
504 [ok]: List invalid list-max-listpack-size config (0 ms)
505 [ok]: List of various encodings (2 ms)
506 [ok]: List of various encodings - sanitize dump (1 ms)
507 [ok]: Unblock fairness is kept while pipelining (0 ms)
508 [ok]: Unblock fairness is kept during nested unblock (1 ms)
509 [ok]: Blocking command accounted only once in commandstats (1 ms)
510 [ok]: Blocking command accounted only once in commandstats after timeout (0
ms)
511 [ok]: Command being unblocked cause another command to get unblocked execution
order test (64 m
512 s)
           [ok]: Blocking timeout following PAUSE should honor the
           timeout (1006 ms)
513 [ok]: Check for memory leaks (pid 7124) (548 ms)
514 [3/3 done]: unit/type/list (12 seconds)
515 Testing solo test
516 [ignore]: Test LPUSH and LPOP on plain nodes over 4GB: large memory flag not
provided
517 [ignore]: Test LINDEX and LINSERT on plain nodes over 4GB: large memory flag
not provided
518 [ignore]: Test LTRIM on plain nodes over 4GB: large memory flag not provided
519 [ignore]: Test LREM on plain nodes over 4GB: large memory flag not provided
520 [ignore]: Test LSET on plain nodes over 4GB: large memory flag not provided
521 [ignore]: Test LSET on plain nodes with large elements under packed_threshold
over 4GB: large m
522 emory flag not provided
           [ignore]: Test LSET splits a quicklist node, and then merge:
           large memory flag not provided
523 [ignore]: Test LSET splits a LZF compressed quicklist node, and then merge:
           large memory flag n
524 ot provided
           [ignore]: Test LMOVE on plain nodes over 4GB: large memory
           flag not provided
525 [ok]: Check for memory leaks (pid 7279) (560 ms)
526 [3/3 done]: list-large-memory (1 seconds)
```

```
527 | Testing solo test
528 | [ignore]: SADD, SCARD, SISMEMBER - large data: large memory flag not provided
529 | [ok]: Check for memory leaks (pid 7305) (514 ms)
530 | [3/3 done]: set-large-memory (1 seconds)
531 |
532 |             The End
533 |
534 | Execution time of different units:
535 |     3 seconds - unit/type/hash
536 |     7 seconds - unit/type/set
537 |    12 seconds - unit/type/list
538 |     1 seconds - list-large-memory
539 |     1 seconds - set-large-memory
540 |
541 | \o/ All tests passed without errors!
542 |
543 | Cleanup: may take some time... OK
544 |
```